

Name: Meghavarshini M

Class: III AIML B

Reg.No: RA2211026050054

Experiment 2 : Conversion of Regular Expression to NFA

AIM: To write a program to convert a Regular Expression into NFA.

ALGORITHM:

- **Start.**
- **Read the regex** input from the user.
- **Use a stack** to construct NFA components:
 - For an **operand (letter/number)**: Create two states with a transition and push to stack.
 - For **concatenation (.)**: Connect two NFAs by merging their end and start states.
 - For **union (|)**: Create a new start and end state, connect both NFAs using ϵ -transitions.
 - For **Kleene star (*)**: Add ϵ -transitions to allow repetition.
- **Extract the final NFA** from the stack.
- **Generate and print** the transition function.
- **Stop.**

PROGRAM:

```
#include <iostream>

#include <stack>

#include <vector>

#include <set>

using namespace std;

struct State {

    int id;

    char symbol;
```

```
State *next1, *next2;
```

```
    State(int id, char symbol = 0, State *n1 = nullptr, State *n2 = nullptr) : id(id),  
    symbol(symbol), next1(n1), next2(n2) {}  
};
```

```
struct NFA {
```

```
    State *start, *end;
```

```
    NFA(State *s, State *e) : start(s), end(e) {}
```

```
};
```

```
class ThompsonConstruction {
```

```
private:
```

```
    int stateCount = 0;
```

```
public:
```

```
    NFA regexToNFA(string regex) {
```

```
        stack<NFA> nfaStack;
```

```
        for (char c : regex) {
```

```
            if (isalnum(c)) {
```

```
                State *s1 = new State(stateCount++);
```

```
                State *s2 = new State(stateCount++);
```

```
                s1->symbol = c;
```

```
                s1->next1 = s2;
```

```
                nfaStack.push(NFA(s1, s2));
```

```

} else if (c == '|') {

    NFA n2 = nfaStack.top(); nfaStack.pop();

    NFA n1 = nfaStack.top(); nfaStack.pop();

    State *s = new State(stateCount++);

    State *e = new State(stateCount++);

    s->next1 = n1.start;

    s->next2 = n2.start;

    n1.end->next1 = e;

    n2.end->next1 = e;

    nfaStack.push(NFA(s, e));

} else if (c == '.') {

    NFA n2 = nfaStack.top(); nfaStack.pop();

    NFA n1 = nfaStack.top(); nfaStack.pop();

    n1.end->next1 = n2.start;

    nfaStack.push(NFA(n1.start, n2.end));

} else if (c == '*') {

    NFA n = nfaStack.top(); nfaStack.pop();

    State *s = new State(stateCount++);

    State *e = new State(stateCount++);

    s->next1 = n.start;

    s->next2 = e;

    n.end->next1 = n.start;

    n.end->next2 = e;

    nfaStack.push(NFA(s, e));

```

```

    }
}

return nfaStack.top();
}

```

```

void printTransitions(State *state, set<int> &visited) {

    if (!state || visited.count(state->id)) return;

    visited.insert(state->id);

    if (state->next1)

        cout << "( q" << state->id << " , " << (state->symbol ? state->symbol : 'E') << " )
-> q" << state->next1->id << endl;

        if (state->next2)

            cout << "( q" << state->id << " , " << (state->symbol ? state->symbol : 'E') << " )
-> q" << state->next2->id << endl;

        printTransitions(state->next1, visited);

        printTransitions(state->next2, visited);

    }

};

```

```

int main() {

    string regex = "ab|c*"; // Example regular expression

    ThompsonConstruction tc;

    NFA nfa = tc.regexToNFA(regex);

    set<int> visited;

```

```
cout << "Transition Function:" << endl;

tc.printTransitions(nfa.start, visited);

return 0;

}
```

INPUT(STRING):

"ab|c*"

OUTPUT:

```
Transition Function
( q8 , E ) -> q6
( q8 , E ) -> q9
( q6 , c ) -> q7
( q7 , E ) -> q6
( q7 , E ) -> q9
```